

Leader Election

Chapter 3

Observations

Election in the Ring

Election in the Mesh

Election in the Hypercube

Election in an arbitrary graph

- Leader Election = Breaking Symmetry

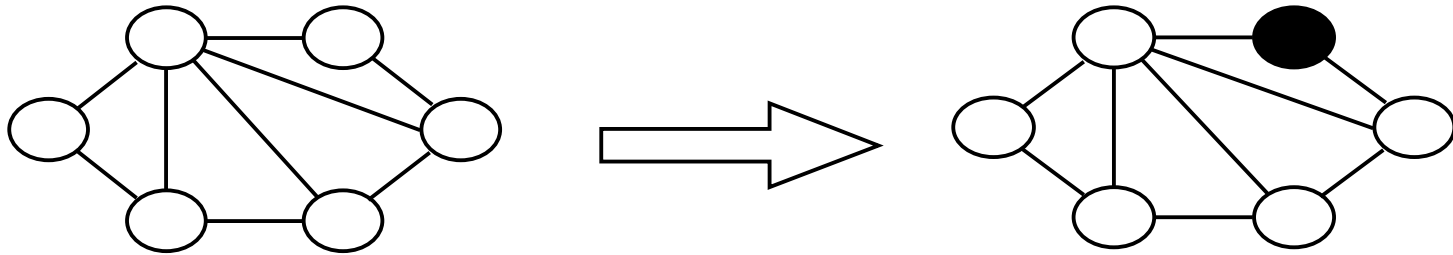
Getting into the **restricted world** with

Unique Initiator

so that

Several Tasks become **much easier**

Election



Theorem [Angluin 80]

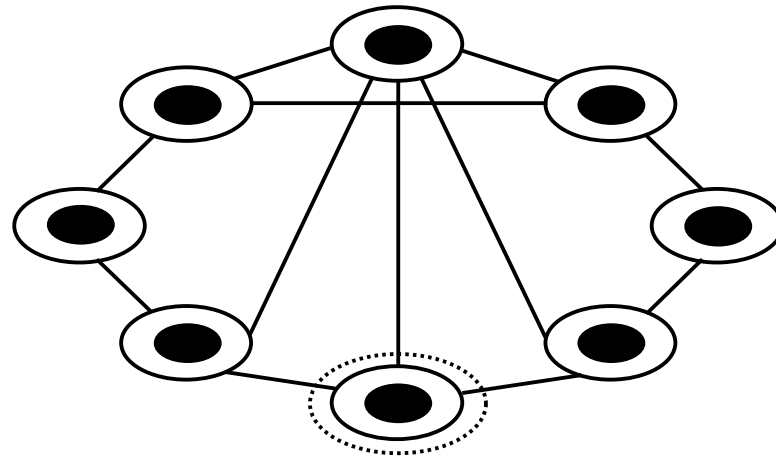
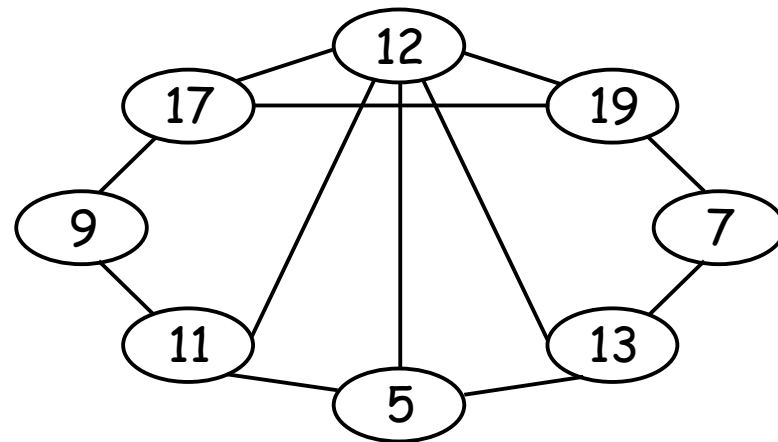
The **election** problem cannot be generally solved if the entities do not have different identities.

Proof. Consider the system where:

- Unique entities
- Same state
- Anonymous
- Synchronous

At each moment, they see the same "world" so they will do the same things --> none can be selected

Observation: with distinct Ids **Minimum Finding** is an election

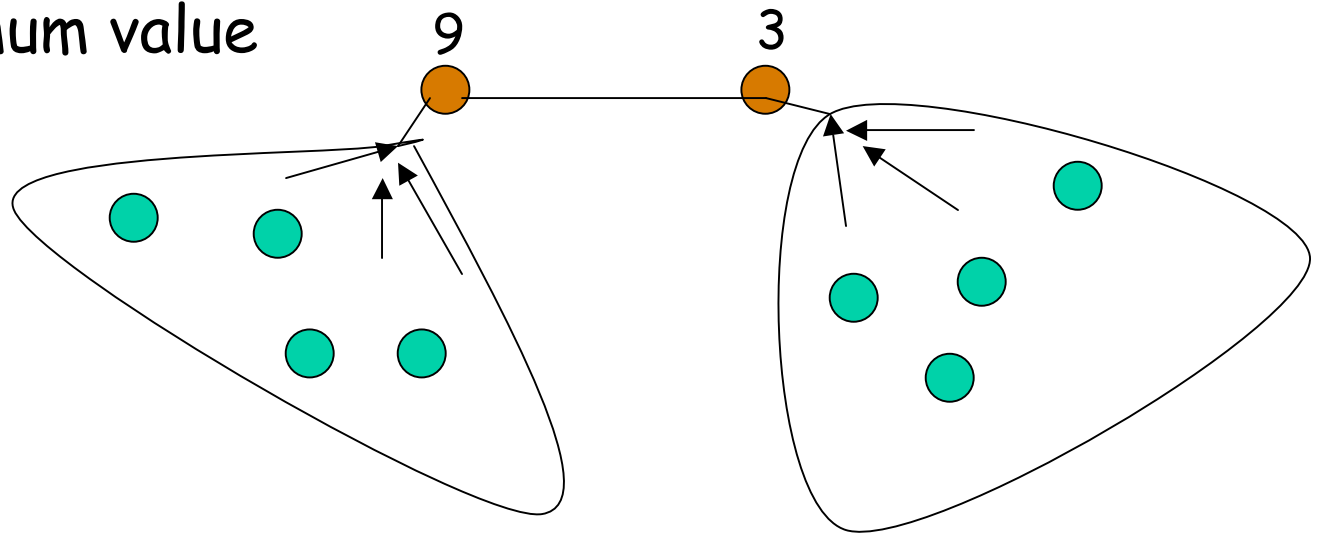


Election in the Tree

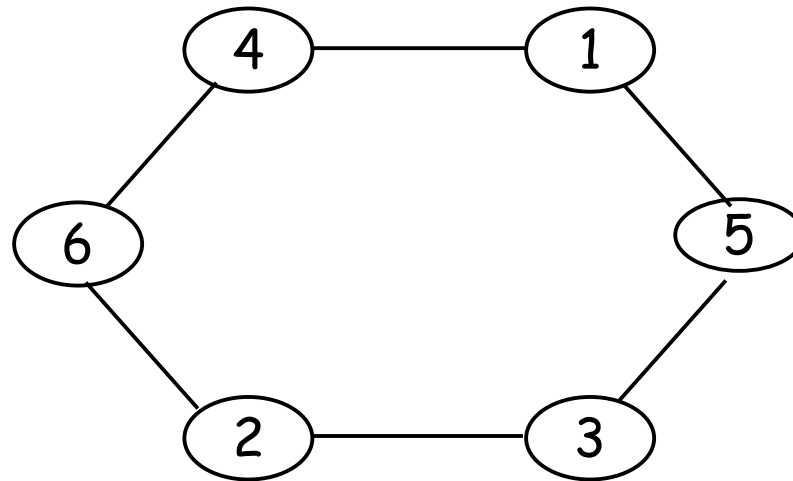
To each node x is associated a distinct identifier $v(x)$

A simple algorithm:

- 1) Execute the saturation technique,
- 2) Choose the saturated node holding the minimum value



Ring

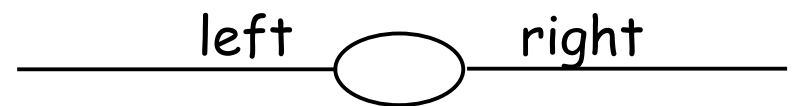


- n entities
- $m = n$ links

- n. of entities = n. of links
- Symmetric topology
- Each entity has two neighbors



When there is sense of direction:



Election Algorithms in Rings

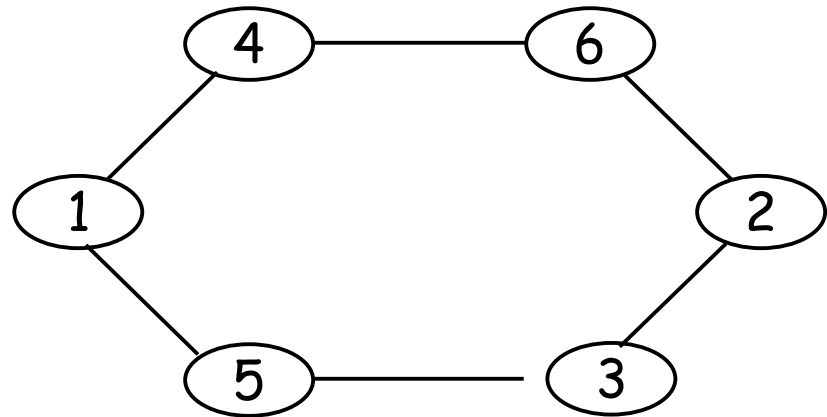
- All the way
- As Far
- Controlled distance
- Electoral stages
 - bidirectional version
- Alternating steps

ELECTING THE MINIMUM

All the way

Basic Idea: Each **id** is fully circulated in the ring.

---> each entity sees **all** identities.



ASSUMPTIONS

- Two versions: unidirectional/bidirectional links.
- Local orientation (i.e. not necessarily a sense of direction)
- Distinct identities.

Termination

When can a node decide to **stop** ?

Answer 1: When he "finally" receives **its** msg back !!

Answer 2: When he receives exactly **$n-1$** other **$id(z)$'s**

Both answers are incorrect!!!

- 1) It assumes **MSG Ordering (FIFO)**
- 2) It assume nodes know **n** (the size of the ring)

Excercise.

Prove that value **n** can be computed by every node with **$O(n^2)$** msg and **$O(n)$** ideal time.

States: $S = \{ASLEEP, AWAKE, FOLLOWER, LEADER\}$
S INIT={ASLEEP};
S_TERM={FOLLOWER, LEADER}.

ASLEEP

Spontaneously

INITIALIZE

become AWAKE

Receiving('`Election'', value, counter)

INITIALIZE;

forward

send('`Election'', value, counter+1)

to other

min:= Min{min, value}

count:= count+1

become AWAKE

INITIALIZE

count:= 0

size:= 1

known:= false

send("Election",id(x),size) to right;

min:= id(x)

AWAKE

Receiving ("Election", value, counter)

If value $\neq$ id(x) then

send ("Election", value, counter+1) to other

min:= MIN {min,value}

count:= count+1

if known = true then

** # of received id's

** Only if know = true, I know n and so I

** can CHECK and TERMINATE

CHECK

endif

else

** I get back mine! so I can know n

ringsize:= counter

known:= true

CHECK

endif

CHECK

if count = ringsize then ** I can STOP here!!

if min = id(x) then

become LEADER

else

become FOLLOWER

endif

endif

MESSAGE Complexity

Each identity crosses each link --> n^2

The size of each message is $\log(id + counter)$

$O(n^2)$ messages

$O(n^2 \log(\text{MaxId}))$ bits

Observations:

1. The algorithm also solves the data collection problem.
2. It also works for unidirectional/bidirectional.

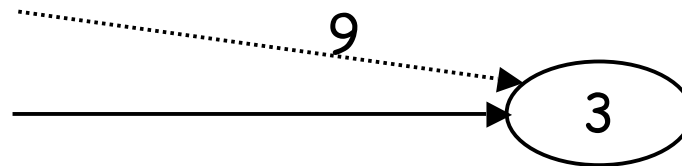
TIME

Time $< 2n$

Proof. Exercise

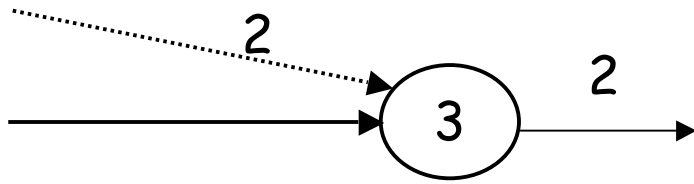
AsFar (as it can)

Basic Idea: It is **not necessary** to send and receive messages with **larger** *ids* than the *ids* that have already been seen.

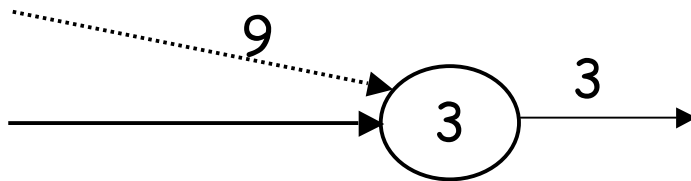


ASSUMPTIONS

- Unidirectional/bidirectional ring
- Different *ids*
- Local orientation

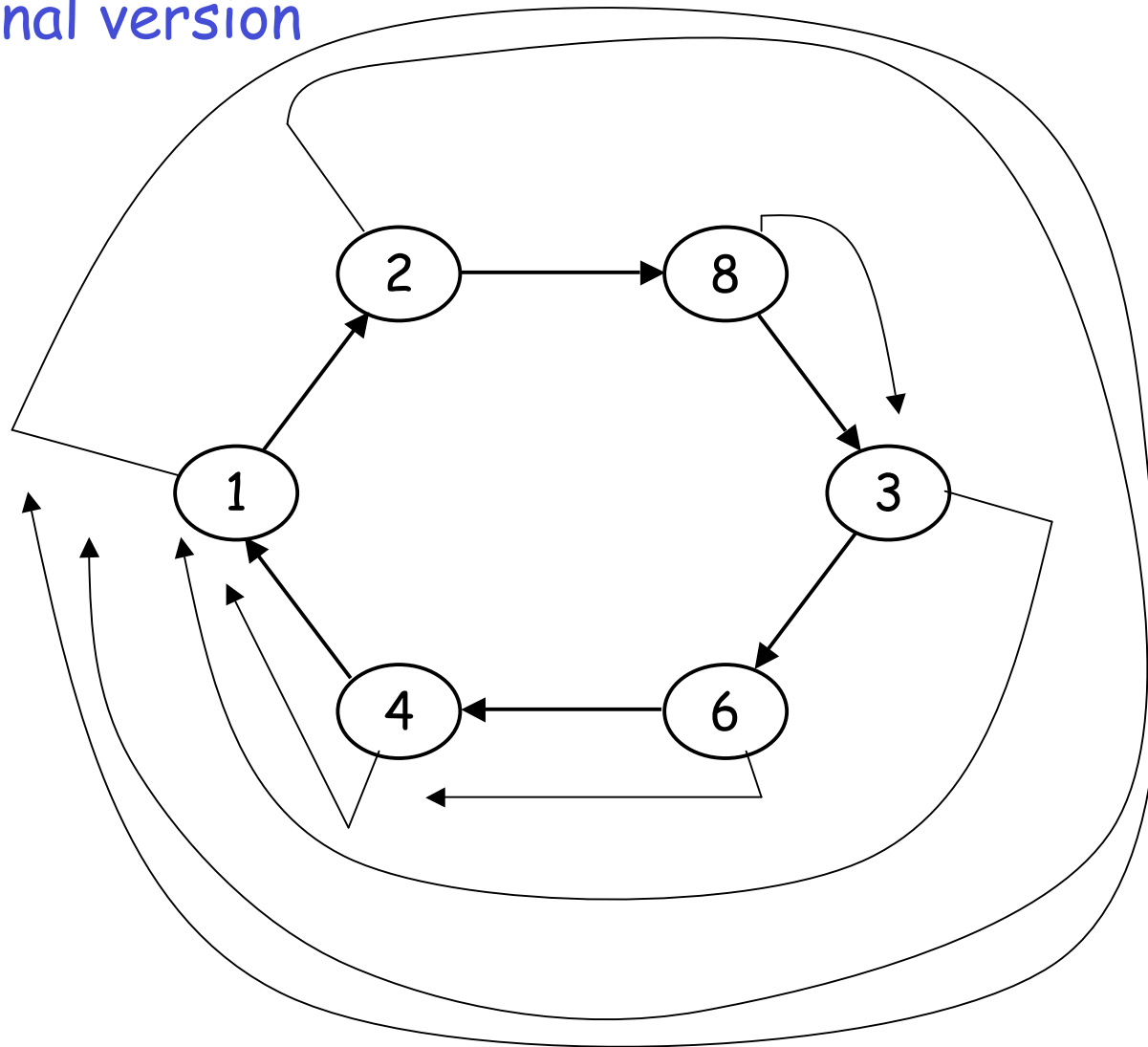


Receiving y **smaller-than** me
 $\text{send}(y)$ to other neighbour



Receiving y **bigger-than** me
 $\text{send}(x)$ to other neighbour
(if not sent already)

unidirectional version



States: $S = \{\text{ASLEEP}, \text{AWAKE}, \text{FOLLOWER}, \text{LEADER}\}$

$S_INIT = \{\text{ASLEEP}\}$

$S_TERM = \{\text{FOLLOWER}, \text{LEADER}\}$

--- unidirectional version

ASLEEP

Spontaneously

send("Election", id(x)) **to right**

min := id(x)

become AWAKE

Receiving("Election", value)

send("Election", id(x)) **to right**

min := id(x)

If value < min then

send("Election", value) **to other**

min := value

endif

become AWAKE

/* this could be avoided if
id(x) > value



AWAKE

Receiving("Election", value)

if value < min then

send("Election", value) **to other**

min:= value

else

If value= min then NOTIFY endif

endif

Receiving(Notify)

send(Notify) **to other**

become FOLLOWER

NOTIFY

send(Notify) **to right**

become LEADER

Correctness and Termination

The **Leader** knows he is the Leader **when** he receives **its own message** back. SO,

He can terminate !!! BUT...

When do the other know ?

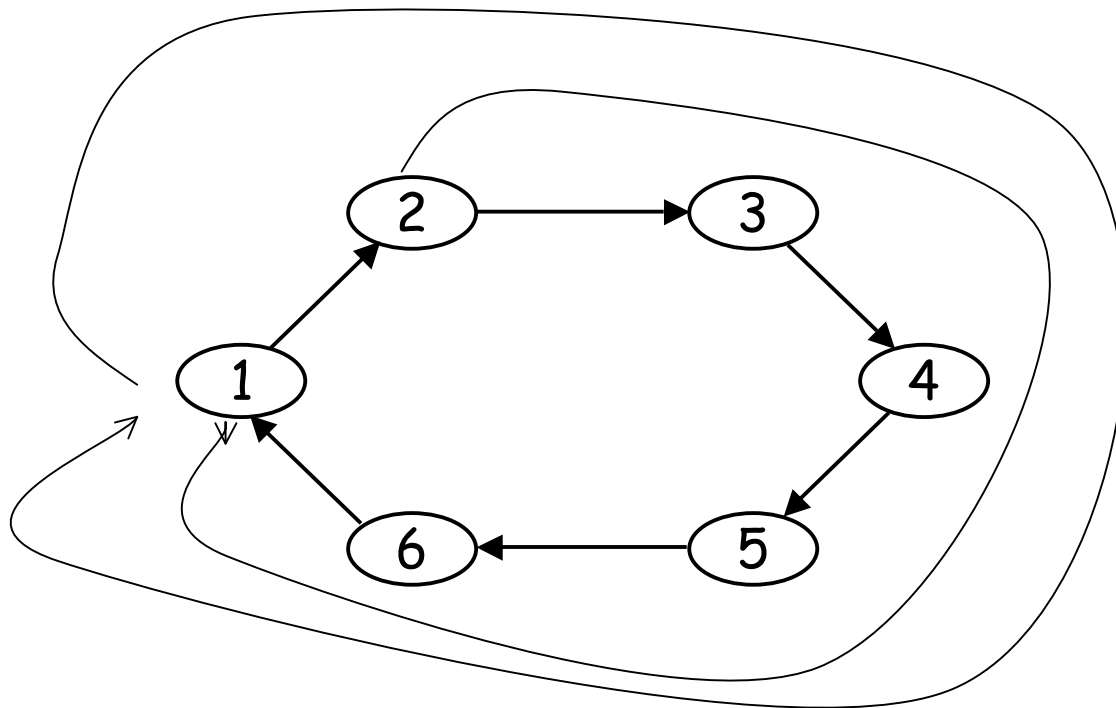
Notification from Leader is necessary !

Message COMPLEXITY

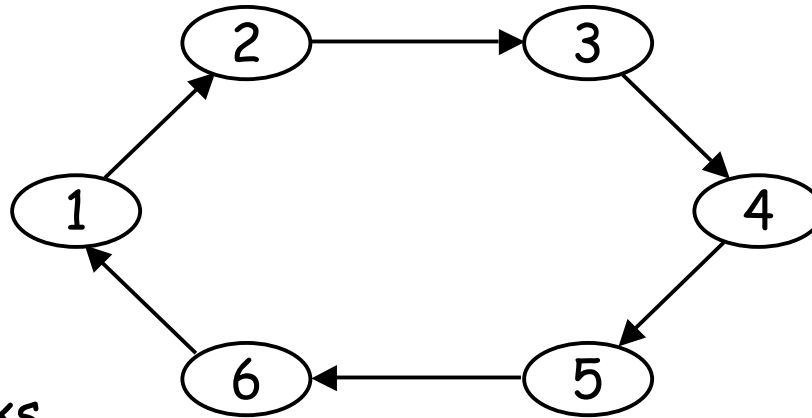
It strongly depends on the ID Configuration over the ring

$$\text{rank}(\text{id}) = (\# \text{ id's less than id}) + 1$$

The Worst-Case situation is :



Worst-Case Complexity

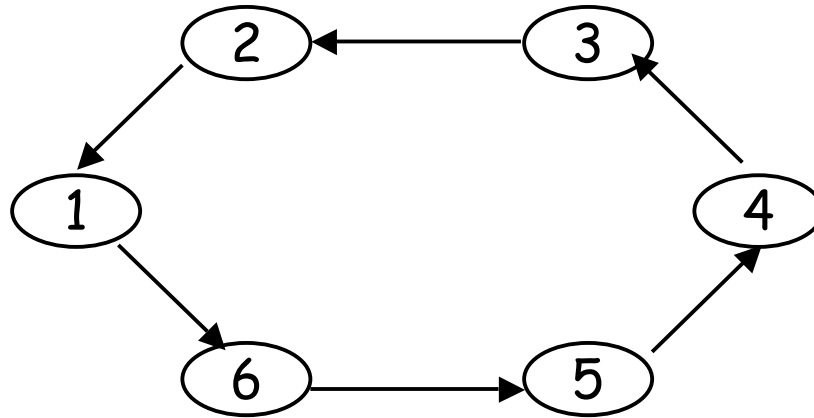


1 ---> n links
2 ---> n - 1 links
3 ---> n - 2 links
... ...
n ---> 1 link

$$n + (n - 1) + (n - 2) + \dots + 1 = \sum_{i=1}^n i = (n+1)(n) / 2$$

Total: $n(n+1)/2 + n$ slightly better than ALL THE WAY
Last n : notification

Best-Case Complexity



1 ----> n links
for all $i \neq 1$ ----> 1 link (--> total = $n - 1$)

Total: $n + (n - 1) + n = O(n)$

Last n : notification